Practice No.: 3 **Feature Viewing / Detection**
Date: 29.07.2026

Aim:

To implement Sobel-based feature edge detection as a post-processing effect on a 3D scene using the three.js web editor.

Algorithm:

1. Menu Bar → Add → Mesh → Box
2. Scene Tree → select Box → Material tab → Type → MeshNormalMaterial
3. Scene Tree → select Scene → Menu Bar → Add → Script
4. Script Editor → paste edge detection code
5. Sidebar → Project tab → Play

Program:

```
function init(event) {
 const rt = new THREE.WebGLRenderTarget(innerWidth, innerHeight);
 const postScene = new THREE.Scene();
 const postCamera = new THREE.OrthographicCamera(-1, 1, 1, -1, 0, 1);

 const mat = new THREE.ShaderMaterial({
  uniforms: {
   tDiffuse: { value: rt.texture },
   resolution: { value: new THREE.Vector2(innerWidth, innerHeight) }
  },
  vertexShader: `
   varying vec2 vUv;
   void main() {
    vUv = uv;
    gl_Position = vec4(position, 1.0);
   }`,
  fragmentShader: `
   uniform sampler2D tDiffuse;
   uniform vec2 resolution;
   varying vec2 vUv;

   float lum(vec2 uv) {
    vec3 c = texture2D(tDiffuse, uv).rgb;
    return dot(c, vec3(0.299, 0.587, 0.114));
   }

   void main() {
    vec2 px = 1.0 / resolution;
    float tl = lum(vUv + px * vec2(-1, 1));
    float t  = lum(vUv + px * vec2(0, 1));
    float tr = lum(vUv + px * vec2(1, 1));
```

```
        float l  = lum(vUv + px * vec2(-1, 0));
        float r  = lum(vUv + px * vec2(1, 0));


float bl = lum(vUv + px * vec2(-1, -1));
        float b  = lum(vUv + px * vec2(0, -1));
        float br = lum(vUv + px * vec2(1, -1));

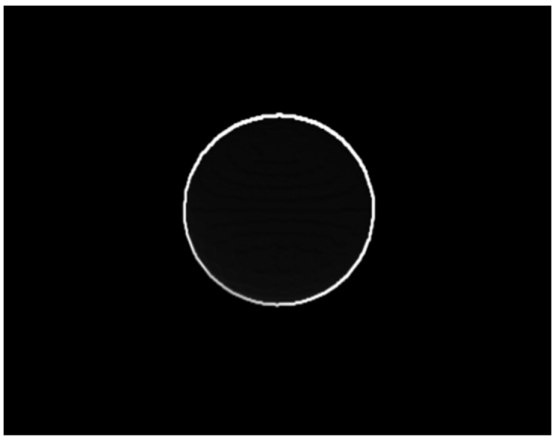
        float gx = -tl - 2.0*l - bl + tr + 2.0*r + br;
        float gy = -tl - 2.0*t - tr + bl + 2.0*b + br;

        gl_FragColor = vec4(vec3(clamp(sqrt(gx*gx + gy*gy), 0.0, 1.0)), 1.0);
      }`
  });

  postScene.add(new THREE.Mesh(new THREE.PlaneGeometry(2, 2), mat));

  const origRender = renderer.render.bind(renderer);
  renderer.render = function (s, c) {
    renderer.setRenderTarget(rt);
    origRender(s, c);
    renderer.setRenderTarget(null);
    origRender(postScene, postCamera);
  };
}
```

Output:



Result:

Thus, output has been obtained for the Feature Edge Detection program.

Practice No.: 7 **Implementation on Virtual Tour**
Date: 26.08.2026

Aim:

To construct a multi-room 3D environment and implement interactive camera navigation to simulate a virtual tour, using the three.js web editor.

Algorithm:

1. Menu Bar → Add → Mesh → Plane (Room 1 floor) → Sidebar Object tab → set Rotation X = −90, Position Z = 6
2. Menu Bar → Add → Mesh → Box (Room 1 walls) → Sidebar Object tab → set Position Z = 6 → Sidebar Material tab → set Color, set Side = Double
3. Menu Bar → Add → Mesh → Plane (Room 2 floor) → Sidebar Object tab → set Rotation X = −90, Position Z = −10
4. Menu Bar → Add → Mesh → Box (Room 2 walls) → Sidebar Object tab → set Position Z = −10 → Sidebar Material tab → set Color, set Side = Double
5. Menu Bar → Add → Light → Ambient
6. Menu Bar → Add → Light → Point → Sidebar Object tab → set Position (0, 3, −4)
7. Viewport → drag/scroll/pan to orient the default camera so it faces from Room 1 toward Room 2 → Scene Tree → select Scene → Menu Bar → Add → Script → paste navigation code
8. Sidebar → Project tab → Play (You must zoom into any one of the boxes you added and THEN ONLY, press PLAY.)

Program:

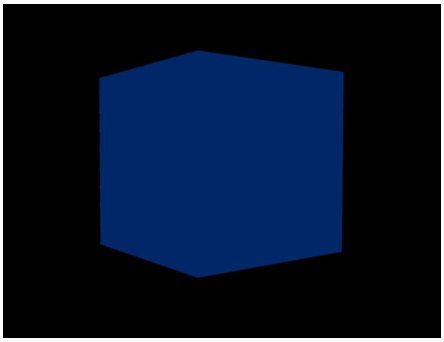
```
let yaw=0, pitch=0, drag=false, px=0, py=0, keys={};

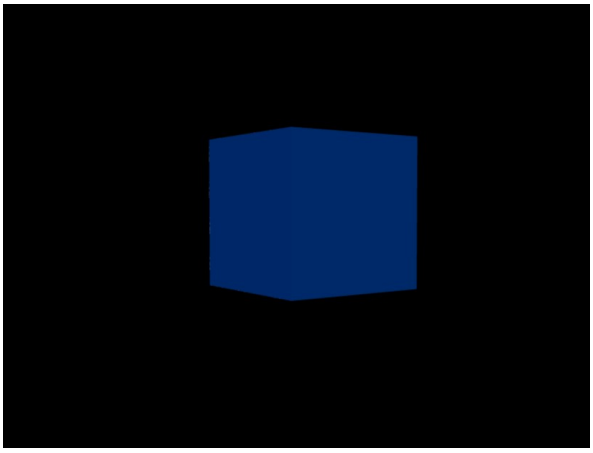
function init(event){
scene.traverse(o => { if(o.material) o.material.side = THREE.DoubleSide; });
}

function pointerdown(e){ drag=true; px=e.clientX; py=e.clientY; }
function pointerup(){ drag=false; }
function pointermove(e){
if(!drag) return;
yaw -= (e.clientX-px)*0.003;
pitch = Math.max(-1,Math.min(1, pitch-(e.clientY-py)*0.003));
px=e.clientX; py=e.clientY;
}
function keydown(e){ keys[e.key.toLowerCase()]=true; }
function keyup(e){ keys[e.key.toLowerCase()]=false; }
function update(event){
const d=new THREE.Vector3(Math.sin(yaw),0,Math.cos(yaw));
const s=new THREE.Vector3(Math.sin(yaw+1.57),0,Math.cos(yaw+1.57));
if(keys['w']) camera.position.addScaledVector(d,-0.12);
if(keys['s']) camera.position.addScaledVector(d,0.12);
```

```
if(keys['a']) camera.position.addScaledVector(s,-0.12);
if(keys['d']) camera.position.addScaledVector(s,0.12);
camera.rotation.set(pitch,yaw,0,'YXZ');
}
```

Output:





Result:

Thus, output has been obtained for the Virtual Tour program.

Practice No.: 8 **Sweeping Coverage of Eye Movements**
Date: 26.08.2026

Aim:

To design a webpage demonstrating sweeping eye movement animation using CSS.

Algorithm:

1. Create two circular div elements styled as eyes using CSS (border-radius: 50%, overflow: hidden).
2. Add a smaller circular div inside each eye to act as the pupil.
3. Define a @keyframes sweep animation that moves the pupil from left to right using translateX.
4. Apply the animation to the pupil with ease-in-out, infinite and alternate so it sweeps back and forth continuously.
5. Display both eyes side by side, centered on the page using flexbox.

Program:

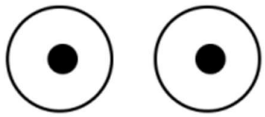
```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Sweeping Eyes</title>
<style>
body {
margin: 0;
height: 100vh;
display: flex;
align-items: center;
justify-content: center;
gap: 40px;
background: #fff;
}


.eye {
width: 100px;
height: 100px;
border: 3px solid #000;
border-radius: 50%;
position: relative;
overflow: hidden;
}
.pupil {
width: 30px;
height: 30px;
background: #000;
border-radius: 50%;
```

```
position: absolute;
top: 35px;
left: 35px;
animation: sweep 2s ease-in-out infinite alternate;
}
@keyframes sweep {
from { transform: translateX(-25px); }
to { transform: translateX(25px); }
}
</style>
</head>


<body>
<div class="eye"><div class="pupil"></div></div>
<div class="eye"><div class="pupil"></div></div>
</body>
</html>
```

Output:



Result:
Two circular eyes appear at the center of the page. In each, the pupil sweeps smoothly from left to right and back, in sync, without ever leaving the eye boundary.

Practice No.: 9 **Implementation on Material Animation**
Date: 02.09.2026

Aim:

To implement Material Design-style animation using CSS transitions on a card and a button.

Algorithm:

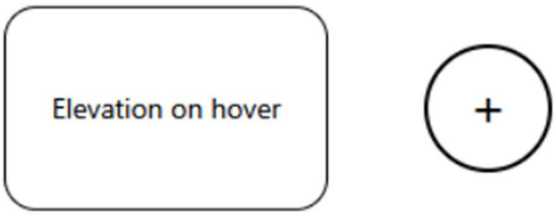
1. Create a card element with a default shadow and border.
2. Add a hover state that lifts the card and increases its shadow.
3. Create a circular floating action button (FAB).
4. Add an active state that morphs the FAB's shape and color on click.
5. Apply CSS transitions to animate both state changes smoothly.

Program:

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Material Animation</title>
<style>
 body {
   margin: 0;
   height: 100vh;
   display: flex;
   align-items: center;
   justify-content: center;
   gap: 48px;
   background: #fff;
 }
 .card {
   width: 160px;
   height: 100px;
   border-radius: 16px;
   border: 1px solid #000;
   display: flex;
   align-items: center;
   justify-content: center;
   font: 14px system-ui;
   transition: .25s;
 }
 .card:hover {
   transform: translateY(-6px);
   box-shadow: 0 10px 24px #0004;
 }
```

```
 .fab {
    width: 64px;
    height: 64px;
    border: 2px solid #000;
    border-radius: 50%;
    background: #fff;
    font-size: 22px;
    cursor: pointer;
    transition: .2s;
  }
  .fab:active {
    border-radius: 20px;
    width: 100px;
    background: #000;
    color: #fff;
  }
</style>
</head>
<body>
  <div class="card">Elevation on hover</div>
  <button class="fab">+</button>
</body>
</html>
```

Output:



Result:

Thus, output has been obtained for the Material Animation program.

Practice No.: 10
Date: 02.09.2026

Understand the Human Physiology and Perception

Aim:

To demonstrate motion parallax, a monocular depth-perception cue, using the three.js web editor.

Algorithm:

1. Menu Bar → Add → Mesh → Box → Sidebar Object tab → set Position Z = 2 → Sidebar Material tab → set Color = red
2. Menu Bar → Add → Mesh → Sphere → Sidebar Object tab → set Position Z = −6 → Sidebar Material tab → set Color = green
3. Menu Bar → Add → Mesh → Cone → Sidebar Object tab → set Position Z = −16 → Sidebar Material tab → set Color = blue
4. Scene Tree → select Scene → Menu Bar → Add → Script
5. Script Editor → paste camera-sweep code
6. Sidebar → Project tab → Play

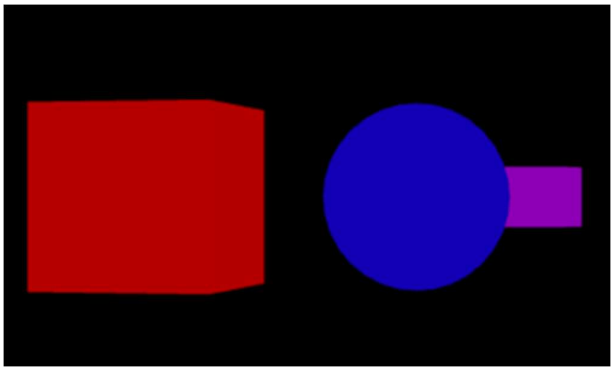
Program:

```
let t = 0;

function init(event) {
  scene.add(new THREE.AmbientLight(0xffffff, 1.5));
  camera.position.set(0, 0, 10);
  camera.lookAt(0, 0, -6);
}

function update(event) {
  t += event.delta * 0.001;
  camera.position.x = Math.sin(t) * 3;
  camera.lookAt(0, 0, -6);
}
```

Output:



Result:

Thus, output has been obtained for the Human Physiology and Perception program.

Practice No.: 12
Date: 16.09.2026

Develop a Simple UI Menu with Images, Canvas, Sprites and Button

Aim:

To develop a simple interactive UI menu using the HTML5 canvas element.

Algorithm:

1. Create a canvas element and get its 2D drawing context.
2. Define a list of menu labels and draw each as a rectangular button.
3. Implement a hit-test function to detect which button the pointer is over.
4. Redraw the canvas on mouse movement to highlight the hovered button.
5. Display an alert with the button's label when clicked.

Program:

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Canvas UI Menu</title>
<style>
  body {
    margin: 0;
    height: 100vh;
    display: flex;
    align-items: center;
    justify-content: center;
    background: #fff;
  }
</style>
</head>
<body>
<canvas id="c" width="240" height="260"></canvas>
<script>
  const ctx = c.getContext('2d');
  const labels = ['Play', 'Settings', 'Profile', 'Exit'];

  function draw(hover = -1) {
    ctx.fillStyle = '#fff';
    ctx.fillRect(0, 0, 240, 260);
    labels.forEach((label, i) => {
      const y = 20 + i * 60;
      ctx.strokeStyle = '#000';
      ctx.fillStyle = i === hover ? '#000' : '#fff';
      ctx.fillRect(20, y, 200, 45);
      ctx.strokeRect(20, y, 200, 45);


ctx.fillStyle = i === hover ? '#fff' : '#000';
      ctx.font = '16px system-ui';
      ctx.textAlign = 'center';
```

```
    ctx.fillText(label, 120, y + 28);
  });
}

function hitTest(x, y) {
  return labels.findIndex((_, i) =>
    x > 20 && x < 220 && y > 20 + i * 60 && y < 65 + i * 60);
}

c.onmousemove = e => draw(hitTest(e.offsetX, e.offsetY));
c.onclick = e => alert(labels[hitTest(e.offsetX, e.offsetY)]);

draw();
</script>
</body>
</html>
```

Output:



Result:

Thus, output has been obtained for the Simple UI Menu program.